

LABORATORIO DI SISTEMI OPERATIVI

Signals

fabio.rossi@unipg.it

075 5855005

IPC (interprocess communication)

- vari strumenti per la comunicazione tra processi:
 - **signal**
 - socket
 - pipe
 - shared memory
 - memory-mapped file
 - etc.

Signals

- ***POSIX Reliable Signals*** (nel seguito *Standard Signals*)
- *Linux* supporta anche i **POSIX Real-Time Signals**
- definiti negli standard **POSIX.1-1990** e **POSIX.1-2001**

Signals: in parole povere....

- forma di *IPC* limitata, nel senso per esempio che non contengono ulteriore informazione (il dato scambiato è il *signal* stesso), *Real-Time* a parte (vedi oltre)
- sono simili ad "*interrupt software*" o, se preferite, a notifiche, che vengono inviati ad un processo (od a un *thread*) al verificarsi di un evento
- gli standard hanno un significato intrinseco e, diciamo così, "tradizionale"...

Signals: cosa accade

- il flusso di esecuzione del processo viene interrotto dal sistema operativo per eseguire la gestione del *signal*:
 - gestione definita dal processo (se lo ha fatto)
 - gestione di default (altrimenti)
- la gestione può essere varia: il processo termina, viene sospeso, viene eseguita una funzione registrata dal processo all'uopo, il signal viene ignorato, etc.

Signals: gestioni di default (da *man 7 signal*)

Signal dispositions

- Se non diversamente specificato ogni *signal* ha una ***disposition*** di default. Le *disposition* di default possono essere:
 - **Term** Default action is to terminate the process.
 - **Ign** Default action is to ignore the signal.
 - **Core** Default action is to terminate the process and dump core (see `core(5)`).
 - **Stop** Default action is to stop the process.
 - **Cont** Default action is to continue the process if it is currently stopped.

Alcuni segnali *standard* in *Linux*

Signal	Standard	Action	Comment
SIGABRT	P1990	Core	Abort signal from abort(3)
SIGALRM	P1990	Term	Timer signal from alarm(2)
SIGBUS	P2001	Core	Bus error (bad memory access)
SIGCHLD	P1990	Ign	Child stopped or terminated
SIGCLD	-	Ign	A synonym for SIGCHLD
SIGCONT	P1990	Cont	Continue if stopped
SIGEMT	-	Term	Emulator trap
SIGFPE	P1990	Core	Floating-point exception
SIGHUP	P1990	Term	Hangup detected on controlling terminal or death of controlling process
SIGILL	P1990	Core	Illegal Instruction
SIGINFO	-		A synonym for SIGPWR
SIGINT	P1990	Term	Interrupt from keyboard
SIGIO	-	Term	I/O now possible (4.2BSD)
SIGIOT	-	Core	IOT trap. A synonym for SIGABRT
SIGKILL	P1990	Term	Kill signal
SIGLOST	-	Term	File lock lost (unused)
SIGPIPE	P1990	Term	Broken pipe: write to pipe with no readers; see pipe(7)
SIGPOLL	P2001	Term	Pollable event (Sys V); synonym for SIGIO
SIGPROF	P2001	Term	Profiling timer expired
SIGUSR1	P1990	Term	User-defined signal 1
SIGUSR2	P1990	Term	User-defined signal 2

Alcuni segnali *standard* in *Linux*

Signal	x86/ARM most others	Alpha/ SPARC	MIPS	PARISC	Notes
SIGHUP	1	1	1	1	
SIGINT	2	2	2	2	
SIGQUIT	3	3	3	3	
SIGILL	4	4	4	4	
SIGTRAP	5	5	5	5	
SIGABRT	6	6	6	6	
SIGIOT	6	6	6	6	
SIGBUS	7	10	10	10	
SIGEMT	-	7	7	-	
SIGFPE	8	8	8	8	
SIGKILL	9	9	9	9	
SIGUSR1	10	30	16	16	
SIGSEGV	11	11	11	11	
SIGUSR2	12	31	17	17	
SIGPIPE	13	13	13	13	
SIGALRM	14	14	14	14	
SIGTERM	15	15	15	15	
SIGSTKFLT	16	-	-	7	
SIGCHLD	17	20	18	18	
SIGCLD	-	-	18	-	
SIGCONT	18	19	25	26	

nota: **SIGUSR1** e **SIGUSR2** sono disponibili per usi applicativi, non hanno un significato "predefinito" per il sistema.

N.B.: la **disposition** di default è per entrambi **Term** !

Sommario

- **Invio**
- *Signal mask*
- *Disposition e Handlers*
 - Interruzione di *system calls* da parte di *signal handlers*
 - considerazioni per gli *handlers*
- *Attesa*
- *Real-time signals*

Signals: sono inviati da...

- dal **sistema operativo**, per comunicare al processo/thread che si è verificato un certo evento (es.: *SIGPIPE*, cfr la lezione sulle *PIPE*)
- **dall'utente** da terminale premendo certi tasti:

- Ctrl+C: SIGINT
- Ctrl+Z: SIGSTP
- Ctrl+\: SIGQUIT

```
fabio@fabioipc:~$ stty -a
speed 38400 baud; rows 46; columns 173; line = 0;
intr = ^C; quit = ^\; erase = ^?; kill = ^U; eof = ^D; eol
= <undef>; eol2 = <undef>; swtch = <undef>; start = ^Q;
stop = ^S; susp = ^Z; rprnt = ^R; werase = ^W; lnext = ^V;
.....
```

- con il comando **kill** dalla shell
- **da un processo/thread all'altro** con apposite funzioni, vedi slide successiva:

Signals: sono inviati da...

The following system calls and library functions allow the caller to send a signal:

- raise(3)** Sends a signal to the calling thread.
- kill(2)** Sends a signal to a specified process, to all members of a specified process group, or to all processes on the system.
- pidfd_send_signal(2)** Sends a signal to a process identified by a PID file descriptor.
- killpg(3)** Sends a signal to all of the members of a specified process group.
- pthread_kill(3)** Sends a signal to a specified POSIX thread in the same process as the caller.
- tgkill(2)** Sends a signal to a specified thread within a specific process. (This is the system call used to implement pthread_kill(3).)
- sigqueue(3)** Sends a real-time signal with accompanying data to a specified process.

Signals: chi li riceve

- *process-directed* o *thread-directed*
 - process-directed:
 - inviati all'intero processo
 - per esempio generati dal *kernel* per motivi diversi da un'eccezione *HW*; o perché inviato con *kill()* o *sigqueue()*
 - ricevuti da uno dei thread che non li bloccano. Se più di uno è il *kernel* che sceglie a quale thread destinarli
 - thread-directed:
 - inviati al thread
 - per esempio perché generati da una specifica istruzione del linguaggio macchina che ha provocato un'eccezione *HW* (per es.: *segmentation fault*, etc.); o inviati con *pthread_kill()*, *tgkill()*

Sommario

- Invio
- ***Signal mask***
- *Disposition e handlers*
 - Interruzione di *system calls* da parte di *signal handlers*
 - *considerazioni per gli handlers*
- Attesa
- *Real-time signals*

Signals: *signal mask*

- un *signal* può essere bloccato, cioè non sarà ricevuto fino a che non verrà sbloccato
- la ***signal mask*** definisce quali *signal* sono bloccati e quali no
- una *signal mask* per ogni *thread* (*per-thread*)
- **SIGKILL** e **SIGSTOP** non possono essere gestiti, ignorati o bloccati
- ereditata dai *child* dopo *fork(2)*; preservata dopo *execve(2)*

Signals: segnali pendenti

- nel tempo tra la generazione e l'invio al processo il *signal* è detto ***pendente***
- può essere pendente perché bloccato
- un thread può ottenere la lista dei segnali pendenti con *sigpending()*
- la lista dei segnali pendenti è resettata (vuota) dopo *fork(2)* ed è preservata dopo *execve(2)*

Signals: funzioni per bloccare signals

- ***sigprocmask()***: per processi *single-threaded*
- ***pthread_sigmask()***: per processi *multithreaded*

Signal: *sigprocmask(2)*

```
#include <signal.h>
int sigprocmask(int how,
                const sigset_t *set,
                sigset_t *oldset);
```

how: *SIG_BLOCK* (The set of blocked signals is the union of the current set and the **set** argument)

SIG_UNBLOCK (The signals in **set** are removed from the current set of blocked signals)

SIG_SETMASK (The set of blocked signals is set to the argument **set**)

set: per manipolare (aggiungere e togliere signal, etc.) variabili di tipo **sigset_t** sono disponibili delle funzioni apposite. Vedi *man 3 sigsetops* e slide successive

oldset: If **oldset** is non-NULL, the previous value of the signal mask is stored in **oldset**

Signal: *pthread_sigmask(3)*

```
#include <signal.h>
int pthread_sigmask(int how,
                    const sigset_t *set,
                    sigset_t *oldset);
```

Argomenti come la *sigprocmask(3)*.

Nota: le funzioni *pthread** possono richiedere di linkare con il flag **-pthread** (viene utilizzata la libreria che contiene le funzioni per i *threads*)

Signal: funzioni per *sigset_t*

```
#include <signal.h>
```

```
int sigemptyset(sigset_t *set);
```

```
int sigfillset(sigset_t *set);
```

```
int sigaddset(sigset_t *set, int signum);
```

```
int sigdelset(sigset_t *set, int signum);
```

```
int sigismember(const sigset_t *set, int signum);
```

Servono per manipolare i singoli elementi di una variabile di tipo *sigset_t*.

(see *man 3 sigsetops*)

Sommario

- Invio
- *Signal mask*
- ***Disposition ed Handlers***
 - Interruzione di *system calls* da parte di *signal handlers*
 - considerazioni per gli *handlers*
- Attesa
- *Real-time signals*

Signals: cambiare la gestione (*disposition*) corrente

- per ogni *signal* ogni processo può cambiare la gestione corrente
- si fa con *sigaction(2)* e *signal(2)* (vedi oltre per il dettaglio di *sigaction(2)*). La seconda è meno portabile, meglio usare l'altra
- tre opzioni:
 - ignorare il signal
 - lasciare/ripristinare la gestione di default del kernel
 - specificare una funzione *handler* che gestisca il segnale
- un *handler* è una funzione definita dal programmatore che il sistema invoca alla ricezione del *signal* (*callback*) in maniera asincrona (il processo/programmatore non sa quando sarà invocata)
- la *disposition* (gestione) di un signal è un attributo del processo condiviso da tutti i suoi threads
- le *disposition* sono duplicate da *fork(2)* nel *child*. Dopo *execve(2)* le *disposition* che prevedono un *handler* sono resettate al default, quelle che prevedono di ignorare rimangono inalterate

Cambiare la *disposition*: *sigaction(2)*

```
#include <signal.h>
int sigaction(int signum,
               const struct sigaction *act,
               struct sigaction *oldact);
```

Usata per variare la gestione di un *signal* da parte di un processo

- ***signum***: specifica il *signal* di cui variare la gestione
- ***act***: *struct* che definisce la nuova gestione
- ***oldact***: *struct* che riceve le informazioni relative alla gestione precedente

Ritorna 0 (successo) o -1 (errore; errno settata)

Registrare *Signal Handlers*: *sigaction*(2)

```
struct sigaction {  
    void      (*sa_handler) (int) ;  
    void      (*sa_sigaction) (int, siginfo_t *, void *) ;  
    sigset_t   sa_mask ;  
    int       sa_flags ;  
    void      (*sa_restorer) (void) ;  
};
```

sa_handler: indica la nuova gestione

- SIG_IGN: ignorato
- SIG_DFL: funzione di default
- puntatore a funzione dell'*handler*. Riceve come unico argomento il numero del *signal* ricevuto

SIGKILL e SIGSTOP non possono essere gestiti, ignorati o bloccati

N.B.: non assegnare contemporaneamente *sa_handler* e *sa_sigaction* (in certe architetture sono oggetto di *union*). In questa sede non parliamo di *sa_sigaction*

Per maggiori informazioni vedere *man sigaction(2)*

Registrare *Signal Handlers*: sigaction()

```
struct sigaction {  
    void      (*sa_handler) (int) ;  
    void      (*sa_sigaction) (int, siginfo_t *, void *) ;  
    sigset_t   sa_mask ;  
    int       sa_flags ;  
    void      (*sa_restorer) (void) ;  
};
```

sa_mask: segnali da mascherare durante l'esecuzione dell'*handler* (in aggiunta a quello che ne ha provocato la chiamata che viene automaticamente bloccato)

Signals: quando è eseguito un handler

- da ***man 7 signal***: *"Whenever there is a transition from kernel-mode to user-mode execution (e.g., on return from a system call or scheduling of a thread onto the CPU), the kernel checks whether there is a pending¹ unblocked² signal for which the process has established a signal handler. If there is such a pending signal, the following steps occur:".....*

vedi slide successiva....

(1): un signal generato si dice "pending" fin quando non viene "consegnato" al processo/thread

(2): un signal generato può essere bloccato; nel caso rimane "pending" fin quando non viene sbloccato

Signals: passi per l'esecuzione dell'handler

Steps (semplicati...):

- il *kernel* esegue operazioni preparatorie per l'esecuzione dell'*handler*:
 - rimuove il signal dalla lista dei pendenti
 - salva lo stato del processo/thread da interrompere (i registri, etc.)
- l'*handler* viene eseguito in *user-mode*
- quando l'*handler* ritorna lo stato del processo/thread viene ripristinato
- il processo/thread che ha ricevuto il signal continua la sua esecuzione

Nota: dal punto di vista del kernel non c'è differenza tra eseguire l'handler o il codice del processo/threads. Nessuna ulteriore "precauzione" viene presa dal kernel e le informazioni sullo stato del processo sono salvate nei registri e nello stack del processo (non nelle aree di memoria del kernel). Per maggiori dettagli vedi **man 7 signal** alla sezione "Execution of signal handlers "

Signal Handlers. Esempio1: *sigaction()*

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
#include<signal.h>
#include<unistd.h>

void SigIntHandler(int iSignalCode) {
    printf("Handler: ricevuto signal %d (%s)\n", iSignalCode, strsignal(iSignalCode));
    return;
}

int main() {
    struct sigaction sa;
    int i;
    memset(&sa, '\0', sizeof(struct sigaction));
    sa.sa_handler = SigIntHandler;    // Provate a variare questa assegnazione
                                     // come specificato nelle due righe seguenti
    //sa.sa_handler = SIG_DFL;
    //sa.sa_handler = SIG_IGN;
    sigaction(SIGINT, &sa, NULL);
    printf("Premere Ctrl+C per eseguire la \"disposition\" di SIGINT\n");
    printf("Se avete definito un handler per Ctrl+C o l'avete ignorato,\n");
    printf("Potete uscire con Ctrl+\\ (provoca l'invio di SIGQUIT)\n");
    for(i = 1; ; i++) {
        sleep(1);
        printf("%d\n", i);
    }
    return EXIT_SUCCESS;
}
```

Sommario

- Invio
- *Signal mask*
- *Disposition e Handlers*
 - Interruzione di *system calls* da parte di *signal handlers*
 - considerazioni per gli *handlers*
- Attesa
- *Real-time signals*

Signals. Esempio2

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include<stdio.h>
#include<string.h>
#include<signal.h>
#include<unistd.h>
#include<errno.h>
#include<stdlib.h>

void SigIntHandler(int iSignalCode) {
    printf("\nHandler: Ricevuto signal %d (%s)\n\n", iSignalCode, strsignal(iSignalCode));
    return;
}

int main() {
    struct sigaction sa;
    unsigned int uiSleepRet;
    memset(&sa, '\0', sizeof(struct sigaction));
    sa.sa_handler = SigIntHandler;
    if(sigaction(SIGINT, &sa, NULL) == -1)
        perror("Main: errore in sigaction");
    printf("Premere Ctrl+C per eseguire l'handler di SIGINT\n");
    printf("Creazione ed apertura fifo...\n");
    mkfifo("mypipe", S_IRUSR|S_IWUSR);
    if(open("mypipe", O_RDONLY) == -1) {
        char s[128]; sprintf(s, "Main: errore in open, errno = %d", errno);
        perror(s); exit(EXIT_FAILURE);
    }
    return EXIT_SUCCESS;
}
```

Signals: interruzione di *SC* bloccate

- nell'*esempio 2* sopra descritto la *open(2)* si interrompe e torna **-1**
- ***errno*** viene settata a ***EINTR***
- in generale.....

Signals: interruzione di SC bloccate

- se un *signal handler* è invocato mentre una SC è bloccata in determinate operazioni allora può verificarsi:
 - la funzione è automaticamente riavviata
 - la funzione fallisce e torna **EINTR**
- ciò dipende da:
 - se in fase di creazione *dell'handler* (*sigaction*) era stato o meno specificato il flag **SA_RESTART**
 - l'interfaccia oggetto della chiamata

Signals: interruzione di SC bloccate

- esempi di SC che se bloccate vengono restartate se *SA_RESTART* è attivo:
 - *read(2)*, *write(2)*, *ioctl(2)*, etc. su "*slow devices*", cioè devices in cui una chiamata I/O può bloccarsi indefinitamente, come *pipe*, *terminale*, *socket*
 - *open(2)* per esempio su una *FIFO*
 - *wait(2)*, *waitpid(2)*
 - *sem_wait(3)*, *sem_timedwait(3)*
 - etc.
- per saperne di più: la sezione "*Interruption of system calls and library functions by signal handlers*" di ***man 7 signal***

Sommario

- Invio
- *Signal mask*
- *Disposition e Handlers*
 - Interruzione di *system calls* da parte di *signal handlers*
 - **considerazioni per gli *handlers***
- Attesa
- *Real-time signals*

Signals: considerazioni per gli handlers

- se più segnali sono pendenti l'ordine in cui sono inviati non è specificato
- gli *standard signal* non si accumulano. Se più istanze dello stesso segnale arrivano mentre esso è bloccato solo un'istanza verrà inviata
- durante l'esecuzione dell'*handler* possono arrivare altri segnali. Questo può generare inconsistenze

Signals: considerazioni per gli handlers

- come visto prima, quando (e se) *l'handler* ritorna il programma riprende da dove aveva lasciato..... ma....
- siamo sicuri che tutto sia consistente?!

Signals: considerazioni per gli handlers

- l'*handler* dovrebbe essere breve e fare il meno possibile
- non tutte le funzioni di libreria possono essere chiamate nell'*handler* senza che possa accadere qualcosa di imprevisto
- *POSIX* definisce un elenco di funzioni che **DEBBONO** essere *async-signal-safe* (vedi *man 7 signal-safety*)
- *i.e:* *printf()* (chiamata nell'esempio di prima) non è in elenco. Non c'è garanzia che non si verifichino comportamenti non previsti

Sommario

- Invio
- *Signal mask*
- *Disposition e handlers*
 - Interruzione di *system calls* da parte di *signal handlers*
 - *considerazioni per gli handlers*
- **Attesa**
- *Real-time signals*

Signals: attesa (generica e sincrona)

- è possibile sospendere l'esecuzione di un processo/*thread* in attesa che arrivi:
 - un segnale gestito con un *handler*
 - un segnale che provochi la terminazione del processo (*disposition* di default)

```
#include <unistd.h>
```

```
int pause(void) ;
```

Attende un signal gestito con handler o che termini il processo. Torna -1 e setta *errno* = **EINTR** se il signal inviato non termina il processo

```
#include <signal.h>
```

```
int sigsuspend(const sigset_t *mask) ;
```

Come *pause()* ma in più rimpiazza temporaneamente la *signal mask* del processo con *mask*. Stesso ritorno di *pause()*

Signals: attendere *signals* specifici

```
#include <signal.h>
```

```
int sigwait(const sigset_t *set, int *sig);
```

- Sospende l'esecuzione ed attende uno dei signal specificati in **set**
- normalmente i segnali da attendere vengono mascherati senza stabilire *handlers* prima di mettersi in attesa
- Accetta il segnale (rimuovendolo dai *pending se mascherato*) e ne torna il numero in *sig*
- ci sono anche **sigwaitinfo(2)** e **sigtimedwait(2)** (vedi *man*)
- realizza una sorta di "attesa sincrona"
- trovate un esempio di **sigwait(2)** più avanti (*esempio 3*)

Signals: Esempio 3

- *SIGALRM* gestito, *SIGINT* mascherato. Attesa sincrona specifica (*sigwait()*)

```
#include<stdio.h>
#include<unistd.h>
#include<string.h>
#include<signal.h>

void SigHandler(int iSignalCode) {
    printf("Handler: ricevuto signal %d: %s\n", iSignalCode, strsignal(iSignalCode));
    return;
}

int main (void) {
    struct sigaction sa;
    int i, signum;
    sigset_t sigset;

    // Gestione con handler di SIGALRM
    memset(&sa, '\0', sizeof(struct sigaction));
    sa.sa_handler = SigHandler;
    sigaction(SIGALRM, &sa, NULL);

    // Mascheramento di SIGINT
    sigemptyset(&sigset);
    sigaddset(&sigset, SIGINT);
    sigprocmask(SIG_BLOCK, &sigset, NULL);

    alarm(3);
    for(i = 1; i <= 4; i++) {
        printf("Main Sigwait n. %d\n", i);
        sigwait(&sigset, &signum);
        printf("Main Sigwait: ricevuto signal %d; %s\n", signum, strsignal(signum));
    }
    printf("Ciao!\n");
    return 0 ;
}
```


Sommario

- Invio
- *Signal mask*
- *Disposition e handlers*
 - Interruzione di *system calls* da parte di *signal handlers*
 - *considerazioni per gli handlers*
- Attesa
- ***Real-time signals***

Real-Time Signals

- Originariamente definiti in POSIX.1b real-time extensions
- Poi inclusi in POSIX.1-2001
- supportati in *Linux* ma non in tutti gli altri sistemi

Real-Time (RT) Signals

- Range definito dalle macro ***SIGRTMIN*** e ***SIGRTMAX***
- questo perché il range dei *RT Signal* può essere variabile a seconda della implementazione dei *thread* della *glibc* a disposizione
- quindi lo sviluppatore deve referenziarli non con un numero assoluto ma con ***SIGRTMIN+n***

Real-Time (RT) Signals

- a differenza degli *Standard Signals* non hanno un significato intrinseco
- tutti possono essere usati per scopi legati all'applicazione
- la disposition di default è terminare il processo

Real-Time (RT) Signals

- più istanze di un *RT signal* possono essere accodate
- invece, per quelli standard, se più istanze dello stesso *signal* sono inviate mentre esso è bloccato solo una è accodata

Real-Time (RT) Signals

- se inviati con *sigqueue(3)* possiamo "abbinare" al *signal* un valore (*int* o *pointer*) che sarà inviato insieme al *signal*
- il processo destinatario può allora registrare un *handler* col flag *SA_SIGINFO* in *sigaction()* e può ottenere questo dato nel campo *si_value* della struttura *siginfo_t*

Real-Time (RT) Signals

- l'ordine in cui i RT Signal sono ricevuti è lo stesso in cui sono stati inviati
- più segnali RT dello stesso tipo sono ricevuti nell'ordine in cui sono stati inviati
- se segnali RT di tipo diverso sono inviati ad un processo sono ricevuti a partire dal numero di segnale più basso
- per quelli standard l'ordine di ricezione dei segnali *pending* non è specificato

Real-Time (RT) Signals

- se per un processo sono pendenti sia standard che *RT signal*, *POSIX* non specifica un ordine di preferenza
- *LINUX*, come altre implementazioni, dà precedenza agli standard

Real-Time (RT) Signals

- Riprendiamo: *"se inviati con sigqueue(3) possiamo abbinare al signal un valore (int o pointer) che sarà inviato col signal".....*

.... Segue *Esempio 4*

Real-Time (RT) Signals: esempio 4

```
#include<stdio.h>
#include<unistd.h>
#include<string.h>
#include<signal.h>
void SigHandler(int iSignalCode, siginfo_t *si, void *p) {
    printf("Handler: ricevuto signal %d: %s\n", iSignalCode, strsignal(iSignalCode));
    printf("Handler: intero passato %d\n", si->si_value.sival_int);
    return;
}
int main (void) {
    struct sigaction sa;
    int i, signum;
    sigset_t sigset;
    union sigval value;

    // Gestione con handler di SIGRTMIN + 1
    memset(&sa, '\0', sizeof(struct sigaction));
    sa.sa_sigaction = SigHandler;
    sa.sa_flags = SA_SIGINFO;
    sigaction(SIGRTMIN + 1, &sa, NULL);

    // Mascheramento di SIGINT
    sigemptyset(&sigset);
    sigaddset(&sigset, SIGINT);
    sigprocmask(SIG_BLOCK, &sigset, NULL);

    sleep(3);
    value.sival_int = 999;
    sigqueue(getpid(), SIGRTMIN + 1, value);
    for(i = 1; i <= 3; i++) {
        printf("Sigwait n. %d\n", i);
        sigwait(&sigset, &signum);
        printf("Sigwait: ricevuto signal %d: %s\n", signum, strsignal(signum));
    }
    printf("Ciao!\n");
    return 0 ;
}
```